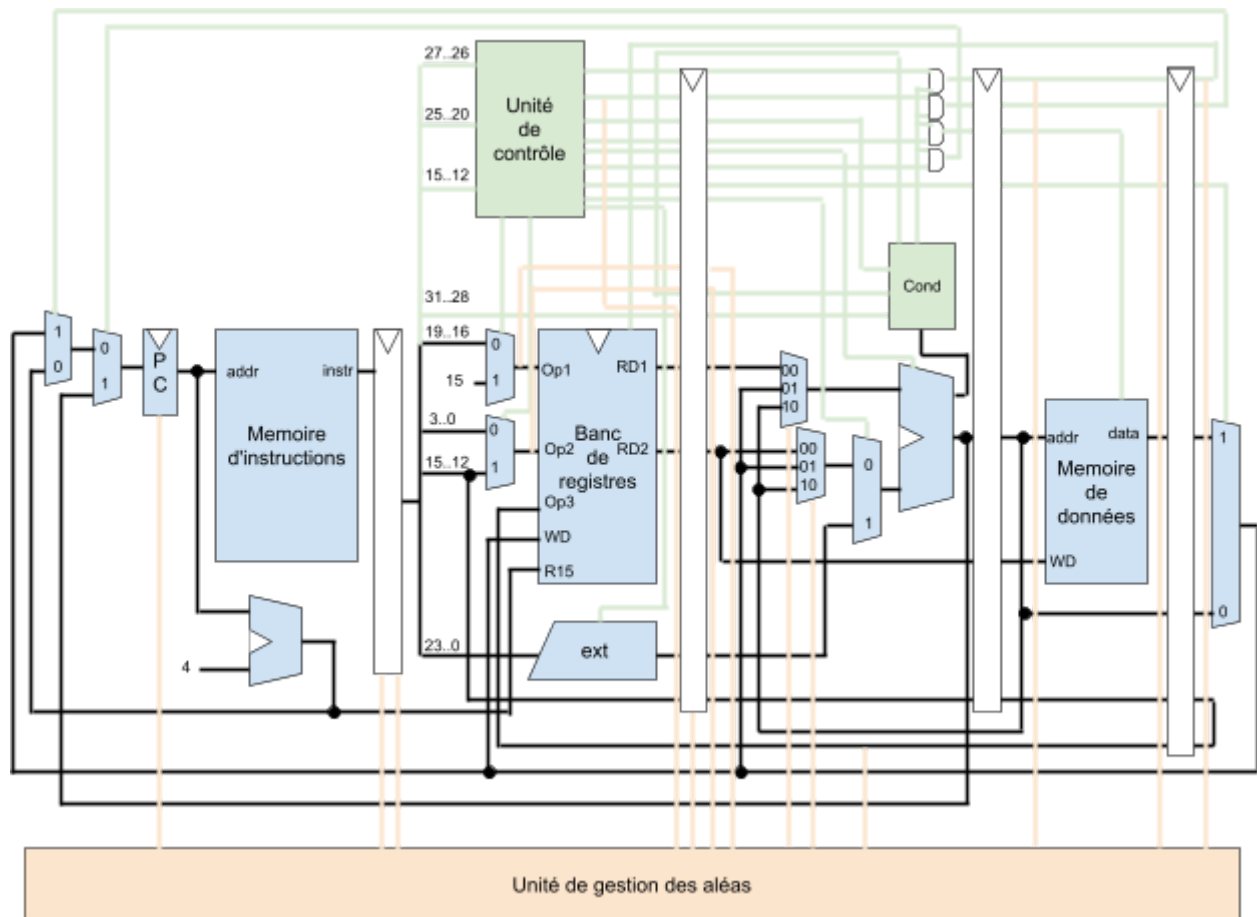


TP MACM (partie 2) – M1 SECIL

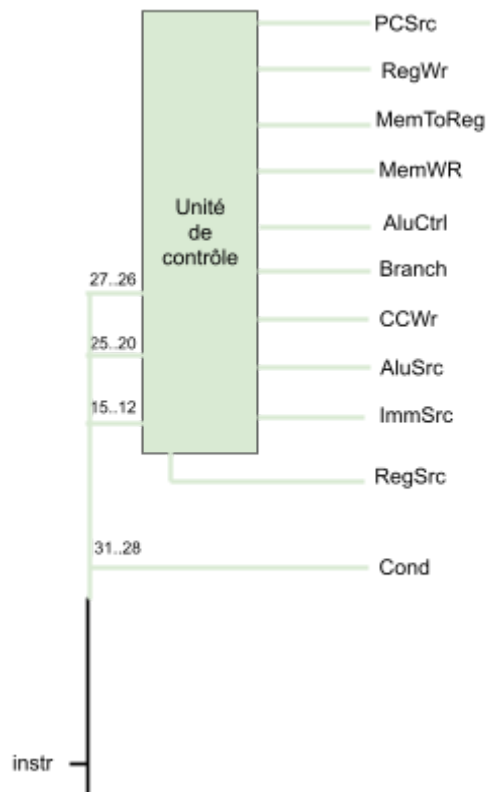
L'objectif de ces 5 séances de TP est de décrire en VHDL un processeur pipeliné dont le jeu d'instructions est basé sur l'ISA ARM (très simplifiée). On commencera par réaliser le chemin de données à partir de composants VHDL fournis, puis on développera la logique de contrôle. Enfin on rajoutera la logique de gestion des aléas de données.



II. Deuxième partie : unité de contrôle et conditions

Nous poursuivons l'implantation de notre processeur, en développant l'unité de contrôle (décodeur) et l'unité de gestion des conditions.

II.1. Décodeur d'instructions/unité de contrôle



Résumé des ports :

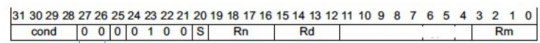
Port	E/S	Taille	Sémantique
instr	E	32 bits	Instruction à décoder
PCSrc	S	1 bit	Signal de sélection de la source du PC (pc_plus_4 ou instruction de calcul à destination de r15)
RegWr	S	1 bit	Signal d'écriture dans le banc de registres
MemToReg	S	1 bit	Signal de sélection du multiplexeur de

			l'étage ER
MemWr	S	1 bit	Signal d'écriture dans la mémoire de données
AluCtrl	S	2 bits	Signal de contrôle de l'opération effectuée par l'ALU
Branch	S	1 bit	Signal de branchement (contrôle le multiplexeur du PC de l'étage FE)
CCWr	S	1 bit	Signal de mise à jour du CPSR (NZCV)
AluSrc	S	1 bit	Signal de sélection entre immédiat et registre pour le second opérande
ImmSrc	S	2 bits	Signal de configuration de l'extension d'immédiats
RegSrc	S	2 bits	Signal de sélection du 1er registre opérande (r15 ou défini dans l'instruction) et du 2nd registre opérande (selon emplacement dans l'instruction)
Cond	S	4 bits	Code de la condition d'exécution

Le signal AluCtrl configure la sortie de l'ALU selon le calcul qui doit être effectué.

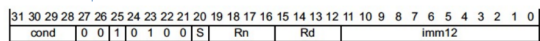
Instr <27..26>	Instr<24..21>	AluCtrl <1..0>
10 (Branch)	XXXX (b)	00 (+)

00 (Calcul)	0100 (ADD)	00 (+)
	0010 (SUB)	01 (-)
	0000 (AND)	10 (Et)
	1100 (ORR)	11 (Ou)
	1010 (CMP)	01 (-)
01 (Memoire)	X0XX (LDR/STR)	00 (+)
	X1XX (LDR/STR)	01 (-)



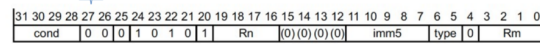
Calcul reg/reg

$Rn+Rm \rightarrow RegSrc<1>=0, RegSrc<0>=0, RegWr=1$



Calcul reg/imm

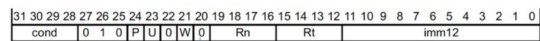
$Rn+Rm \rightarrow RegSrc<1>=0, RegSrc<0>=0, RegWr=1$
 $ImmSrc<1..0>=00$



CMP

$Rn-Rm \rightarrow RegSrc<1>=0, RegSrc<0>=0$
 $RegWr=0$

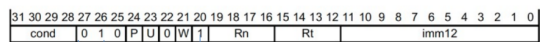
AluSrc



STR

$Rn+imm \rightarrow RegSrc<0>=0$
 $Rt \text{ à écrire en mémoire } \rightarrow RegSrc<1>=1$
 $RegWr=0$

$ImmSrc<1..0>=01$



LDR

$Rn+imm \rightarrow RegSrc<0>=0$
 $RegWr=1$



Branchement

$R15+ext(imm24) \rightarrow RegSrc<0>=1$
 $ImmSrc<1..0>=01, RegWr=0$

Branch, $RegSrc<0>$

Instr <27..26>	Instr <25>	Instr <20>	Branch	MemTo Reg	MemWr	AluSrc	ImmSrc	RegWr	RegSrc
00 reg/reg	0	X	0	0	0	0	XX	1	00
00 (CMP)	0	1	0	0	0	0	XX	0	00
00 reg/imm	1	X	0	0	0	1	00	1	X0
01 (LDR)	X	1	0	1	0	1	01	1	X0
01 (STR)	X	0	0	X	1	1	01	0	10
10 (B)	X	X	1	0	0	1	10	0	X1

PCSrc vaut 1 lorsque la cible d'une instruction de calcul ou d'un chargement de données est r15 :

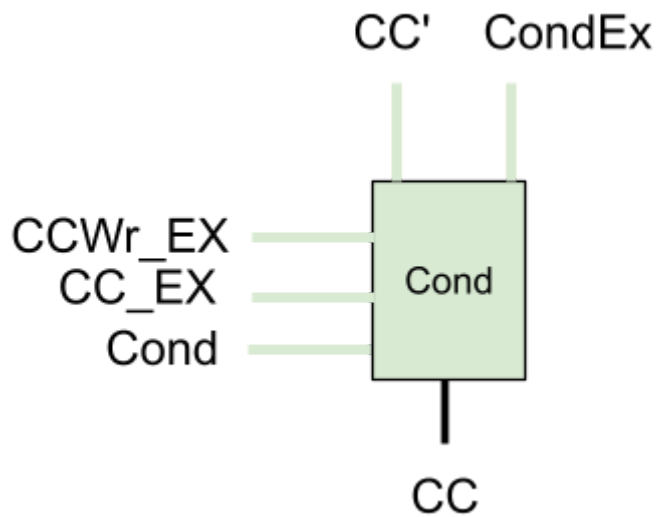
Instr <27..26>	Instr <25>	Instr <20>	Instr <15..12>	PCSrc
00 reg/reg	0	X	1111	1
00 reg/reg	0	1	XXXX (autre que 1111)	0
00 reg/imm	1	X	1111	1
00 reg/imm	1	X	XXXX (autre que 1111)	0
01 (LDR)	X	1	1111	1
01 (LDR)	X	1	XXXX (autre que 1111)	0
01 (STR)	X	0	XXXX	0
10 (B)	X	X	XXXX	X

CCWr est à 1 lorsque le bit S est à 1 et que l'instruction est un calcul (cela comprend aussi les CMP) :

Instr <27..26>	Instr <25>	Instr <20>	CCWr
00 reg/reg	0	0	0
00 reg/reg	0	1	1
00 reg/imm	1	0	0
00 reg/imm	1	1	1

01 (LDR)	X	1	0
01 (STR)	X	0	0
10 (B)	X	X	0

II.2 Unité de gestion des conditions



L'unité de gestion des conditions permet de mettre à jour les bits N, Z, C, V lorsque l'instruction le demande (bit S) et de générer le signal CondEx qui spécifie si l'instruction couramment dans l'étage EX doit être exécutée ou non. Pour ce faire, la valeur sauvegardée des bits N, Z, C, V (CC_EX) est utilisée

Port	E/S	Taille	Sémantique
Cond	E	4 bits	Code de la condition de l'instruction dans l'étage EX
CC_EX	E	4 bits	Valeurs couramment sauvegardées de N, Z, C, V
CC	E	4 bits	Valeurs de N, Z, C, V calculées au cycle

			courant par l'ALU
CCWr_EX	E	1 bit	Signal de mise à jour des bits N, Z, C, V
CC'	S	4 bits	Nouvelle valeur des bits N, Z, C, V (soit égale à l'ancienne valeur sauvegardée, soit à la nouvelle valeur calculée)
CondEx	S	1 bit	Contrôle d'exécution: 1-> l'instruction doit être exécutée, 0 -> l'instruction n'est pas exécutée.

On a donc CC' qui vaut CC si CCWr_EX vaut 1 et que CondEx vaut 1 (c'est -à -dire si l'instruction courante s'exécute et que le bit S est à 1), et CC_EX sinon.

Pour le calcul de CondEx on utilise le tableau suivant:

Opcode	Condition	Sémantique	Calcul
0000	EQ	Egal/Egal à 0	Z set
0001	NE	Non égal	Z clear
0010	CS/HS	Carry set / unsigned higher or same	C set
0011	CC/LO	Carry clear / unsigned lower	C clear
0100	MI	Négatif	N set
0101	PL	Positif	N clear
0110	VS	Overflow	V set
0111	VC	Pas d'overflow	V clear
1000	HI	Unsigned Higher	C set et Z clear
1001	LS	Unsigned lower or same	C clear ou Z set
1010	GE	Signed greater or	N == V

		equal	
1011	LT	Signed less than	N != V
1100	GT	Signed greater than	Z clear et N == V
1101	LE	Signed less than or equal	Z set ou N != V
1110	AL	Toujours vrai	1

II.3 Instanciation des unités de contrôle et de condition dans le pipeline

